

도커컴포즈

- 도커컴포즈는 여러 개의 Docker 컨테이너를 관리하는 도구로서 Docker 데스크탑 설치시 기본으로 설치된다.
- 한 번의 명령어로 여러 개의 컨테이너를 한번에 실행하거나 종료할 수 있다.
- 단일 서버에서 여러 개의 컨테이너를 하나의 서비스로 정의해 컨테이너의 묶음으로 관리할 수 있는 작업 환경을 제공하는 관리 도구이다.
- 시스템 구축과 관련된 명령어를 하나의 텍스트 파일(정의 파일, Compose File)에 기재해 한 번의 명령으로 시스템 전체를 실행하거나 종료한다.
- docker-compose.yml 파일에서 같은 네트워크 설정을 하지 않아도 기본적으로 같은 Docker Compose 프로젝트에 정의된 서비스끼리는 자동으로 같은 사용자 정의 네트워크에 속하게 된다. 그래서 명시적으로 **networks:** 항목을 작성하지 않아도 서로 통신이 가능하다.

Docker Compose를 사용하는 이유

1. 여러 개의 컨테이너를 관리하는 데 용이하다.

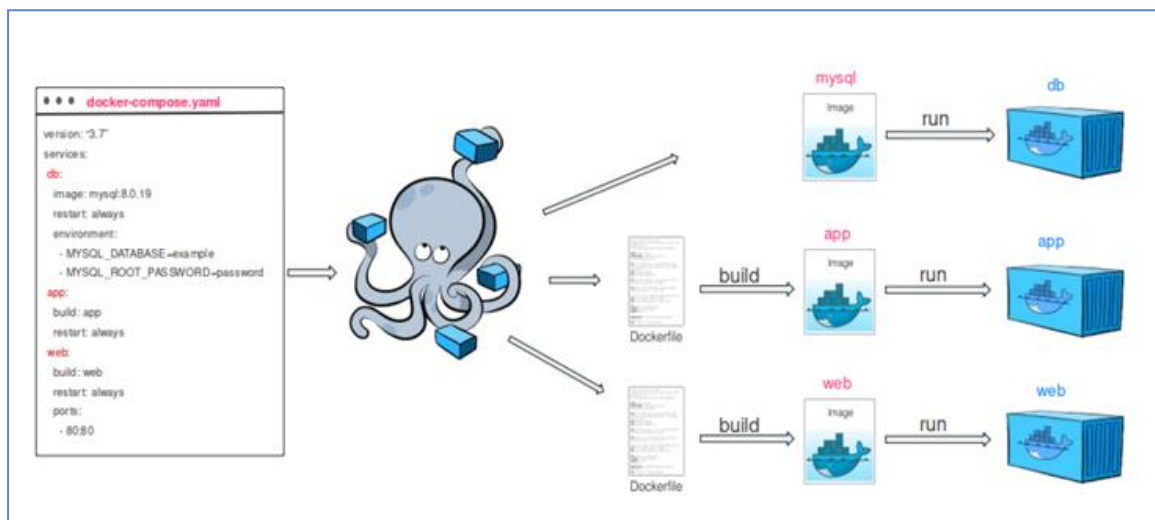
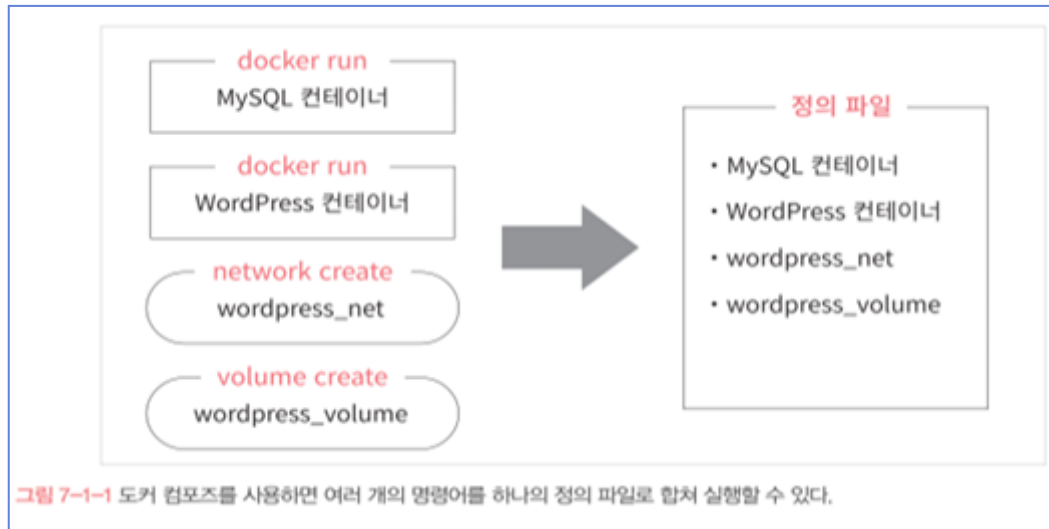
여러 개의 컨테이너로 이루어진 복잡한 애플리케이션을 한 번에 관리할 수 있게 해준다.
여러 컨테이너를 하나의 환경에서 실행하고 관리하는 데 도움이 된다.

2. 복잡한 명령어로 실행시키던 걸 간소화 시킬 수 있다.

이전에 MySQL 이미지를 컨테이너로 실행시킬 때 아래와 같은 명령어를 실행시켰다.

```
docker run -e MYSQL_ROOT_PASSWORD=password123 -p 3306:3306 -v  
/Users/jaeseong/Documents/Develop/docker-mysql/mysql_data:/var/lib/mysql -d mysql
```

너무 복잡하다! Docker Compose 를 사용하면 위와 같이 컨테이너를 실행시킬 때마다 복잡한 명령어를 입력하지 않아도 된다. 단순히 **docker compose up** 명령어만 실행시키면 된다.



YAML(YAML Ain't a Markup Language)포맷의 정의 파일

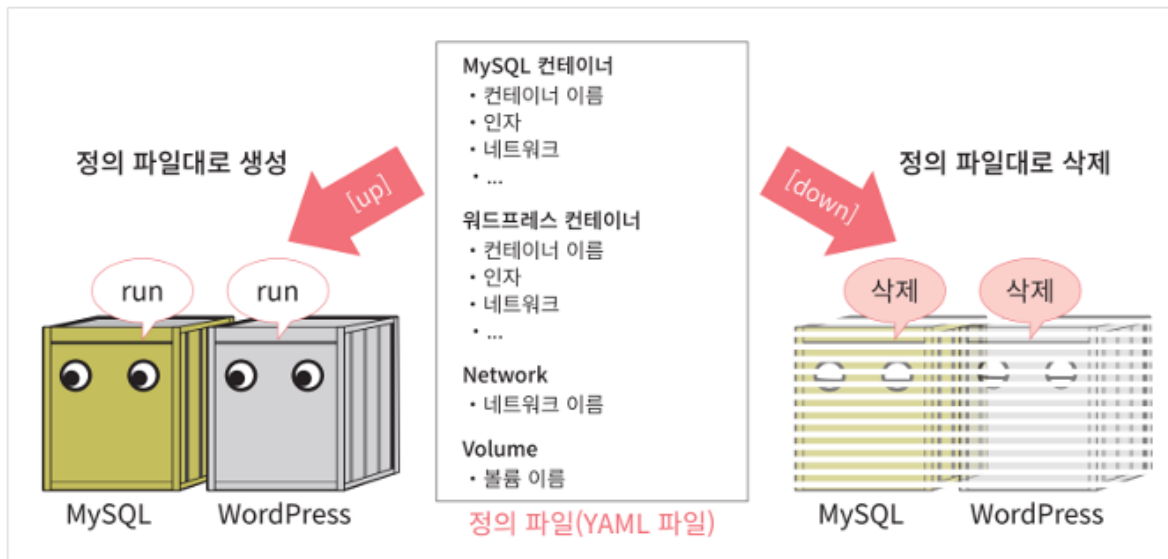


그림 7-1-2 도커 컴포즈는 시스템의 모든 정보를 정의 파일에 기재한다.

- **up 명령** : 정의 파일(docker-compose.yml)의 내용대로 이미지를 내려 받고 컨테이너를 생성 및 실행, 네트워크나 볼륨도 정의 가능
- **down 명령** : 컨테이너와 네트워크를 정지 및 삭제
- **stop 명령** : 컨테이너와 네트워크를 삭제 없이 종료만 진행

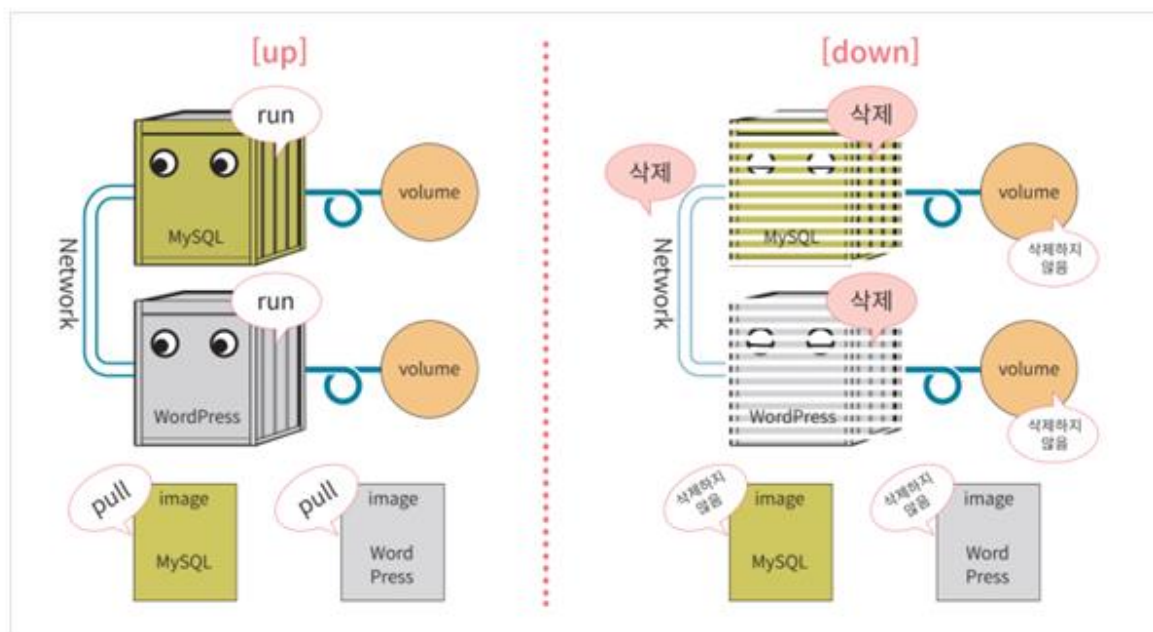


그림 7-1-3 up 커맨드로 생성 및 시작, down 커맨드로 정지 및 삭제한다.

도커컴포즈와 Dockerfile 스크립트 차이점

- 도커컴포즈 : 컨테이너와 주변 환경 및 네트워크와 볼륨까지 만들 수 있다.
- Dockerfile스크립트 : 이미지를 만들기 위한 것으로 네트워크나 볼륨은 만들 수 없다

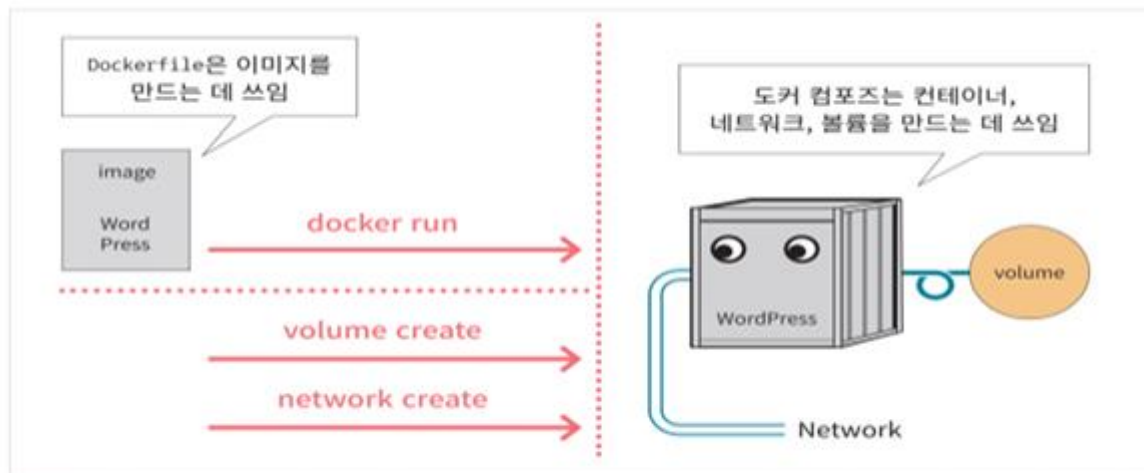


그림 7-1-4 Dockerfile 스크립트와 도커 컴포즈의 차이점

도커컴포즈와 쿠버네티스의 차이점

1)쿠버네티스(Kubernetes) : 도커 컨테이너 관리 도구

Kubernetes란?

Kubernetes(K8s)는 컨테이너 오케스트레이션 도구

즉, 여러 개의 Docker 컨테이너를 대규모로 관리, 배포, 확장, 복구하는 시스템이다.

핵심 역할

- 수백~수천 개의 컨테이너를 자동으로 관리
- 로드 밸런싱, 자동 복구, 자동 확장(autoscaling) 기능 제공
- 여러 서버(노드)에 걸쳐 컨테이너를 효율적으로 배포
- 선언형 설정 파일(yaml) 기반 관리

2)도커컴포즈(docker-compse) : 컨테이너 생성 및 삭제, 관리 기능은 없음

Docker vs Kubernetes 비교 요약

구분	Docker	Kubernetes
핵심 목적	컨테이너 생성 및 실행	여러 컨테이너의 관리 및 자동화
규모	단일 호스트	다중 노드(클러스터)
관리 단위	컨테이너	Pod(컨테이너 묶음), Node, Cluster
자동화 기능	없음 (수동 관리)	있음 (자동 배포, 복구, 확장 등)
사용 예	개발용, 소규모 서비스	운영 환경, 대규모 서비스
관계	컨테이너를 만드는 도구	컨테이너들을 관리하는 시스템

도커컴포즈 설치

파이썬으로 작성된 프로그램이다.

Docker Desktop에는 도커컴포즈 함께 설치되어 있다.

Spring Boot 애플리케이션을 도커컴포즈를 이용하여 리눅스 기반의 AWS의 EC2에 배포하려면 직접 도커컴포즈는 설치해야 한다.

```
sudo apt install -y python3 python3-pip
sudo pip3 install docker-compose
```

#확인

```
docker-compose --version
docker-compose -v
```

```
Docker Compose version v2.31.0-desktop.2
```

docker compose up -d

YAML 파일에 정의된 서비스 생성 및 시작

docker compose ps

현재 실행중인 서비스 상태 표시

docker compose build

현재 실행중인 서비스의 이미지만 빌드

docker compose logs

실행 중인 서비스의 로그 표시

docker compose down

YAML 파일에 정의된 서비스 종료 및 제거

[도커컴포즈 사용법]

- 정의 파일명 : **docker-compose.yml or compose.yml**(Docker Compose V2부터 지원됨)

- 도커 엔진에 의해 수행

사람이 입력하는 수동 명령을 컴포즈가 대신 전달하는 개념

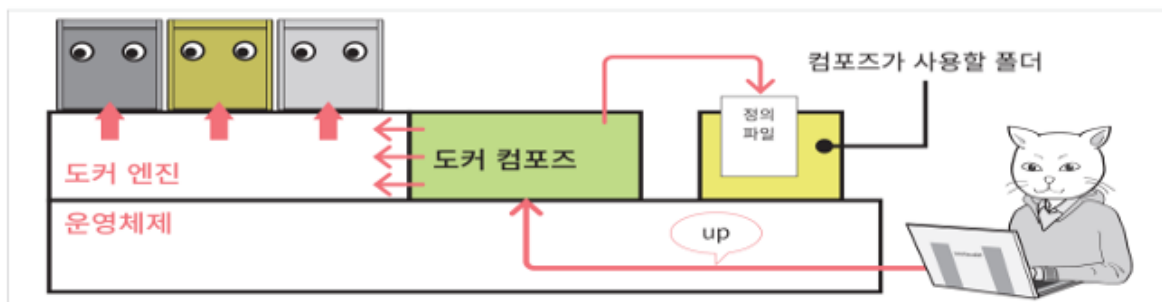


그림 7-2-3 도커 컴포즈 사용하기

- 서비스와 컨테이너

도커컴포즈에서 서비스란 컨테이너가 모인 것을 의미

일반적으로 컨테이너와 서비스는 동일한 개념으로 봐도 됨

- 컴포즈 파일 작성 방법
- 주 항목 -> 이름 추가 -> 설정의 순으로 진행

```
services:    # 컨테이너 관련 정보
networks :   # 네트워크 관련 정보
volumes:     # 볼륨 관련 정보
```

- 첫 번째 줄에 도커 컴포즈 버전 작성
- 주 항목 services, networks, volumes 아래에 설정 내용 작성
- 이름 추가 시 주 항목보다 한 단 들여쓰기
- 이름 뒤에는 반드시 ':' 사용
- : 문자 뒤에 한 줄에 이어서 설정이 필요한 경우 : 뒤에 공백문자가 1개가 반드시 필요
- 이름 뒤에 여러 개의 설정이 필요한 경우 줄을 바꿔 들여쓰기와 함께 '하이픈(-)+공백문자'와 함께 설정 내역 작성
- 항목 간의 상하 관계는 공백을 사용한 들여쓰기로 정함
- 들여쓰기는 같은 수의 배수만큼의 공백을 사용
- # 은 주석 처리
- 문자열은 '' 또는 ""를 이용
- 탭 문자는 사용 못함

```
version:"3"
services:
  컨테이너_이름1:
    image:네트워크_이름
  networks:
    네트워크_이름1:
```

공백 두 개 →
공백 네 개 →
공백 여섯 개 →

- docker-compose.yml 템플릿

```

version: "1"
services:
  컨테이너_이름1:
    image: 이미지_이름
    networks:
      - 네트워크_이름
    ports:
      - 포트_설정
  컨테이너_이름2:
    image: 이미지_이름
    ...
networks:
  네트워크_이름1:
    ...
volumes:
  볼륨_이름1:
  볼륨_이름2:
  ...

```

자주 나오는 정의 내용

항목	docker run 커맨드의 해당 옵션 또는 인자	내용
image	이미지 인자	사용할 이미지를 지정
networks	--net	접속할 네트워크를 지정
volumes	-v, --mount	스토리지 마운트를 설정
ports	-p	포트 설정
environment	-e	환경변수 설정
depends_on	없음	다른 서비스에 대한 의존관계를 정의
restart	없음	컨테이너 종료 시 재시작 여부를 설정

restart의 설정값

설정값	내용
no	재시작하지 않는다.
always	항상 재시작한다.
on-failure	프로세스가 0 외의 상태로 종료됐다면 재시작한다.
unless-stopped	종료 시 재시작하지 않음. 그 외에는 재시작한다.

depends_on : 컨테이너 생성 순서나 연동 여부를 정의한다.

예) depends_on:

- mysqledu

현재 컨테이너를 mysqledu 컨테이너를 생성한 다음에 생성하라는 의미이다.

- 도커 컴포즈 정의 파일명 : **docker-compose.yml**

다른 파일명이거나 다른 디렉토리에 존재하는 경우에는 -f 와 함께 지정 가능

docker-compose -f **정의파일이름** up 옵션

옵션 항목

옵션	내용
-d	백그라운드로 실행
--no-color	화면 출력 내용을 흑백으로 함
--no-deps	링크된 서비스를 실행하지 않음
--force-recreate	설정 또는 이미지가 변경되지 않더라도 컨테이너를 재생성
--no-create	컨테이너가 이미 존재할 경우 다시 생성하지 않음
--no-build	이미지가 없어도 이미지를 빌드하지 않음
--build	컨테이너를 실행하기 전에 이미지를 빌드
--abort-on-container-exit	컨테이너가 하나라도 종료되면 모든 컨테이너를 종료
-t, --timeout	컨테이너를 종료할 때의 타임아웃 설정. 기본은 10초.
--remove-orphans	컴포즈 파일에 정의되지 않은 서비스의 컨테이너를 삭제
--scale	컨테이너의 수를 변경

사용 예)

```
docker-compose -f docker-compose2.yml up -d
```

현재 작업 디렉터리에 존재하는 `docker-compose.yml` 을 컴포즈 정의 파일로 사용할 때에는 따로 컴포즈 파일을 지정하지 않아도 됨

```
docker-compose up -d
```

- 컨테이너와 네트워크 삭제 명령

```
docker-compose -f 정의_파일_이름 down 옵션
```

옵션 항목

옵션	내용
--rm 종류	삭제 시에 이미지도 삭제한다. 종류를 all로 지정하면 사용했던 모든 이미지가 삭제된다. local로 지정하면 커스텀 태그가 없는 이미지만 삭제한다.
-v, --volumes	volumes 항목에 기재된 볼륨을 삭제한다. 단, external로 지정된 볼륨은 삭제되지 않는다.
--remove-orphans	컴포즈 파일에 정의되지 않은 서비스의 컨테이너도 삭제한다.

- 컨테이너만 종료하는 명령

```
docker-compose -f 정의_파일_이름 stop
```

- docker-compose 주요 명령 목록

커맨드	내용	커맨드	내용
up	컨테이너를 생성하고 실행한다.	create	컨테이너를 생성한다.
down	컨테이너와 네트워크를 종료하고 삭제한다.	restart	컨테이너를 재실행한다.
ps	컨테이너 목록을 출력한다.	pause	컨테이너를 일시 정지한다.
config	컴포즈 파일을 확인하고 내용을 출력한다.	unpause	컨테이너의 일시 정지를 해제한다.
port	포트 설정 내용을 출력한다.	rm	종료된 컨테이너를 삭제한다.
logs	컨테이너가 출력한 내용을 화면에 출력한다.	build	컨테이너에 사용되는 이미지를 빌드 혹은 재빌드한다.
start	컨테이너를 시작한다.	pull	컨테이너에 사용되는 이미지를 내려받는다.
stop	컨테이너를 종료한다.	scale	컨테이너의 수를 지정한다.
kill	컨테이너를 강제 종료한다.	events	컨테이너로부터 실시간으로 이벤트를 수신한다.
exec	명령어를 실행한다.	help	도움말 화면을 출력한다.
run	컨테이너를 실행한다.		

docker-compose.yml 파일을 이용한 컨테이너 배포실습

실습 > ex04_myPC_docker_compose폴더

todoList 프로젝트 ROOT.war를 Dockerfile을 이용하여 이미지로 만들어 docker-compose.yml의 서비스 이미지로 사용한다.

init.sql파일을 이용하여 테이블생성과 초기레코드를 설정한다.

#Dockerfile

Tomcat 10 이미지를 기반으로 컨테이너를 생성

FROM tomcat:10

현재 디렉토리의 ROOT.war 파일을 Tomcat의 webapps 폴더에 복사하여 배포

COPY ROOT.war /usr/local/tomcat/webapps/

#docker-compose.yml

```

1  version: '3.8' # Docker Compose 파일의 버전
2
3  >Run All Services
4  services: # 여러 컨테이너(서비스)를 정의하는 영역
5    >Run Service
6    mysql: # MySQL 데이터베이스 서비스 정의
7      image: mysql:8.4 # 사용할 MySQL 이미지와 버전
8      container_name: my_sql # 생성될 컨테이너의 이름
9      environment: # 환경변수 설정
10       MYSQL_ROOT_PASSWORD: "1234" # root 계정 비밀번호 설정
11       MYSQL_DATABASE: "web_basic" # 초기 생성할 데이터베이스 이름
12       TZ: "Asia/Seoul" # 컨테이너의 시간대를 서울로 설정
13     volumes: # 볼륨 설정 (데이터 유지 및 초기화용)
14       - mysql_vol:/var/lib/mysql # DB 데이터를 영속적으로 저장
15       - ./db-init:/docker-entrypoint-initdb.d # SQL 스크립트를 자동 실행할 폴더 마운트
16       - /etc/localtime:/etc/localtime:ro # 호스트의 시간 설정을 컨테이너에 반영 (읽기 전용)
17     ports:
18       - "3306:3306" # 호스트의 3306 포트를 컨테이너의 3306 포트에 연결
19     networks:
20       - web_net # web_net 네트워크에 연결
21
22   >Run Service
23   tomcat: # 톰캣 웹 서버 서비스 정의
24     build: . # 현재 디렉토리에 있는 Dockerfile을 사용하여 이미지 빌드
25     image: my_tomcat:latest # ← 내가 원하는 이미지 이름으로 지정
26     container_name: my_tomcat # 생성될 톰캣 컨테이너의 이름
27     ports:
28       - "80:8080" # 호스트의 80포트를 컨테이너의 8080포트에 연결
29     networks:
30       - web_net # web_net 네트워크에 연결
31     depends_on:
32       - mysql # mysql 컨테이너가 먼저 실행되어야 tomcat 실행됨
33
34   volumes: # 데이터 저장소 정의 (컨테이너 재시작 시 데이터 유지)
35     mysql_vol: # MySQL 데이터 볼륨
36
37   networks: # 사용자 정의 네트워크 정의
38     web_net: # mysql과 tomcat이 통신할 내부 네트워크

```

volumes: - ./db-init:/docker-entrypoint-initdb.d 이 의미

- 왼쪽 : ./db-init → 호스트(로컬) 컴퓨터의 폴더를 의미
- 오른쪽 : /docker-entrypoint-initdb.d → MySQL 컨테이너 내부의 특별한 초기화 스크립트 폴더

즉, 로컬의 db-init 폴더를 MySQL 컨테이너 안의 초기화 전용 폴더에 마운트함

/docker-entrypoint-initdb.d란?

MySQL 공식 Docker 이미지에서는 컨테이너 최초 실행 시, 이 폴더에 있는 .sql, .sql.gz, .sh 파일들을 자동으로 실행해줌

- 예: init.sql, create_table.sql, load_data.sh 등

단, MySQL 데이터가 비어 있을 때(처음 컨테이너 생성 시)만 한 번 실행됨.
이미 데이터가 있는 상태에서는 이 폴더의 스크립트는 무시된다.

db-init 폴더 안의 파일명은 꼭 init.sql이어야 하나?

- *.sql, *.sql.gz, *.sh 형식이면 되고, 파일 이름은 상관없음
 - 01_schema.sql
 - create_user.sql
 - init.sh (셸스크립트)

파일 이름을 숫자나 알파벳 순서로 정리하면 실행 순서 제어가 가능하다
(예: 01_create_db.sql, 02_insert_data.sql).

- /etc/localtime:/etc/localtime:ro 설정은 왜 필요?

컨테이너의 시간대를 호스트(EC2나 내 컴퓨터)의 시간대와 맞추기 위해 사용하는 설정
컨테이너는 기본적으로 UTC 시간대를 사용한다. 그래서 로그나 DB의 시간 등이 호스트 시간과 다르게 나올 수 있다.

예를 들어:

- 호스트 시간: 2025-05-03 14:00 (Asia/Seoul)
 - 컨테이너 시간 (기본): 2025-05-03 05:00 (UTC)
- DB 에 저장된 timestamp 나 로그에 9 시간 차이 발생!

:ro를 왜?

읽기 전용(read-only)을 의미

- 시간대를 맞추기 위한 목적이므로, 컨테이너가 이 파일을 수정하지 못하도록 보호함
- 실수나 보안상 이유로 변경 방지.

#docker-compose.yml파일 실행하기

cmd창에서 이미지빌드(--build), 컨테이너 생성과 실행(up) , 백그라운드실행(-d)

docker-compose up --build -d

http://localhost/



페이지 14 / 22

인스턴스 시작

▼ 키 페어(로그인) 정보

키 페어를 사용하여 인스턴스에 안전하게 연결할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

키 페어 이름 - **test-key** 새 키 페어 생성

▼ 네트워크 설정 정보

네트워크 정보
vpc-029bf3f33d1054705

서브넷 정보
기본 설정 없음(가용 영역의 기본 서브넷)

퍼블릭 IP 자동 할당 정보
활성화

프리 티어 허용 범위를 벗어나는 경우 추가 요금이 적용됩니다.

방화벽(보안 그룹) 정보

☒ 보안 그룹 생성 ☐ 기존 보안 그룹 선택

다음 규칙을 사용하여 'launch-wizard-2'(이)라는 새 보안 그룹을 생성합니다.

☒ 다음에서 SSH 트래픽 허용 → 22번 원격접속
인스턴스 연결에 도움 위치 무관 0.0.0.0/0

☐ 인터넷에서 HTTPS 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

☒ 인터넷에서 HTTP 트래픽 허용 → tomcat 80 으로 접속예정
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

⚠ 소스가 0.0.0.0/0인 규칙은 모든 IP 주소에서 인스턴스에 액세스하도록 허용합니다. 알려진 IP 주소의 액세스만 허용하도록 보안 그룹을 설정하는 것이 좋습니다.

▼ 스토리지 구성 정보

1x 30 GiB gp3 루트 볼륨, 3000IOPS, 암호화되지 않음 스토리지 30GB설정

요약

인스턴스 개수 정보
1

소프트웨어 이미지(AMI)
Canonical, Ubuntu, 24.04, amd64... 더 보기
ami-0d5bb3742db8fc264

가상 서버 유형(인스턴스 유형)
t2.small

방화벽(보안 그룹)
새 보안 그룹

스토리지(볼륨)
1개의 볼륨 - 30GiB

❗ 프리 티어: AWS 계정을 개설한 첫해에 프리 티어 AMI, 매월 750시간의 퍼블릭 IPv4 주소 사용량, 30GiB의 EBS 스토리지, 2백만 I/O, 1GB의 스냅샷 및 100GB의 인터넷 대역폭과 함께 사용하면 매월 750시간의 t2.micro 인스턴스 사용량(또는 t2.micro를 사용할 수 없는 경우 t3.micro)을 제공받습니다.

취소 인스턴스 시작 미리 보기 코드

인스턴스에서 확인

인스턴스 (1) 정보

인스턴스를 속성 또는 (case-sensitive) 태그로 찾기 모든 상태

Name	인스턴스 ID	인스턴스 상태	인스턴스 유형	상태 검사	경보 상태	가용 영역	퍼블릭 IPv4 DNS	퍼블릭 IPv4 ...	탄력적 IP	IPv6 IP
docker-test	i-0e9109200ad9aeb4b	실행 중	t2.small	2/2개 검사 통과	경보 보기	ap-northeast-2a	ec2-43-202-2-110.ap-n...	43.202.2.110	-	-

#MobaXterm으로 EC2 원격 접속하기

인스턴스 속성 또는 (case-sensitive) 태그로 찾기 모든 상태

Name	인스턴스 ID	인스턴스 상태	인스턴스 유형	상태 검사	경보 상태	가용 영역	퍼블릭 IPv4 DNS	퍼블릭 IPv4 ...
docker-test	i-0e9109200ad9aeb4b	실행 중	t2.small	2/2개 검사 통과	경보 보기	ap-northeast-2a	ec2-43-202-2-110.ap-n...	43.202.2.110

SSH

Basic SSH settings

Remote host: 43.202.2.110 Specify username: ubuntu Port: 22

Advanced SSH settings

☒ X11-Forwarding ☒ Compression Remote environment: Interactive shell

Execute command:

☐ Do not exit after command ends

SSH-browser type: SFTP protocol ☐ Follow SSH path (experimental)

☒ Use private key: D:\2025년_privateStudy\20050328 Expert SSH settings

Execute macro at session start: <none>

생성한 키페어연결 OK Cancel

AWS EC2 Ubuntu에 Docker를 설치하기 위해 아래의 명령어를 순서대로 진행한다.

Docker설치 전체흐름(보안 + 공식 저장소 등록 + 의존성 해결)

순서	단계명	핵심 목적
1	필수 패키지 설치	HTTPS 및 저장소 관리 도구 준비
2	GPG 키 추가	보안 인증 (신뢰할 수 있는 서명 확인)
3	Docker 저장소 등록	공식 Docker 패키지 소스 연결
4	apt 업데이트	새 저장소 반영
5	Docker 설치	Docker 엔진 설치 완료

#최신 패키지를 설치하기 위해 패키지 목록을 업데이트 – 최신버전 정보 받아오기

#명령어

```
sudo apt update
```

#Docker를 설치하려면 몇 가지 필수 패키지를 먼저 설치해야 한다.

: Docker 설치에 의존되는 도구들을 미리 준비

: Docker는 HTTPS 저장소를 사용하므로 아래 도구들이 없으면 Docker 설치가 불가능

: Docker 저장소를 추가할 수 있는 환경을 만드는 단계

패키지 이름	역할
apt-transport-https	APT가 HTTPS를 통해 패키지를 다운로드할 수 있게 함
ca-certificates	HTTPS 인증서를 검증하는 데 필요
curl	웹에서 파일을 다운로드할 수 있는 도구
software-properties-common	add-apt-repository 명령어를 사용할 수 있게 해줌

#명령어

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

#Docker의 공식 GPG 키 등록

: Docker에서 제공하는 패키지가 정상적인지(위조되지 않았는지) 검증하기 위한 디지털 서명 키 (GPG) 를 등록한다. 보안을 위해 APT는 서명된 패키지만 설치하도록 되어 있어서, GPG 키를 미리 등록해 놓아야 APT가 Docker를 신뢰하고 설치한다.

: Docker가 진짜 Docker가 맞는지 확인하는 단계

#명령어

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee  
/etc/apt/trusted.gpg.d/docker.asc
```

#Docker의 공식 저장소를 APT 소스에 추가

: APT가 Docker 패키지를 다운로드할 **공식 저장소 URL**을 등록

즉, apt가 Docker를 어디서 받을지 알려주는 설정 단계

: Ubuntu 기본 저장소에는 Docker의 **최신 버전이 없기 때문에** Docker 공식 저장소를 등록해야 최신 버전의 docker-ce를 설치할 수 있다.

#명령어

```
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu  
$(lsb_release -cs) stable"
```

#새로운 Docker 저장소가 추가되었으므로, 패키지 목록을 다시 업데이트

위 단계에서 **Docker 저장소를 새로 등록했기 때문에**, 다시 apt update로 **해당 저장소의 패키지 정보도 불러와야** 한다. docker-ce가 이제 APT에서 검색 가능해졌으므로, 목록을 다시 갱신해 줘야 설치 가능하다.

#명령어

```
sudo apt update
```

#Docker를 설치

: docker-ce는 Docker의 Community Edition이다.

: Docker 데몬, CLI 등 핵심 도구들을 함께 설치한다.

#명령어

```
sudo apt install docker-ce
```

#Docker가 자동으로 시작되지만, 확인을 위해 서비스를 시작할 수 있다.

#명령어

```
sudo systemctl start docker
```

#Docker 서비스가 정상적으로 실행되고 있는지 확인

#명령어

```
sudo systemctl status docker
```

#Docker 설치가 완료되었는지 확인하려면 버전을 확인

#명령어

```
docker --version
```

```
ubuntu@ip-172-31-12-26:~$ docker --version  
Docker version 28.1.1, build 4eba377
```

#현재 사용자로 root 권한을 부여 받은 후, root 사용자로 전환한다

#명령어

```
sudo su
```

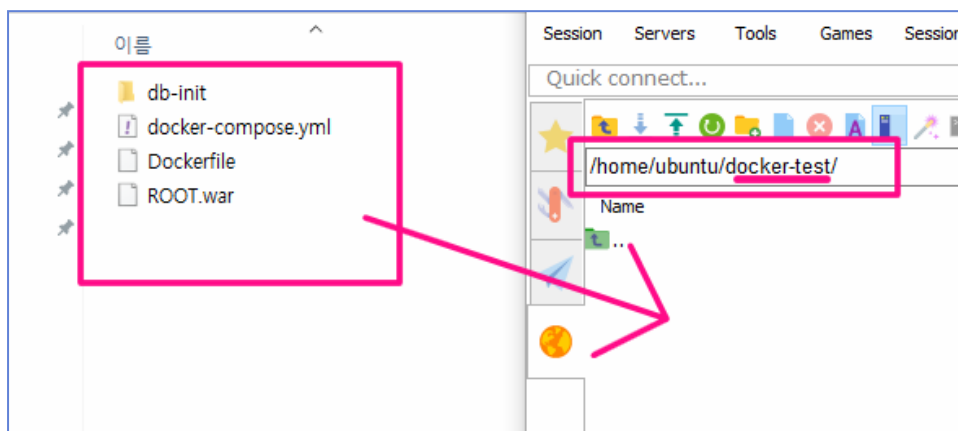
```
ubuntu@ip-172-31-12-26:~$ sudo su  
root@ip-172-31-12-26:/home/ubuntu#
```

폴더생성

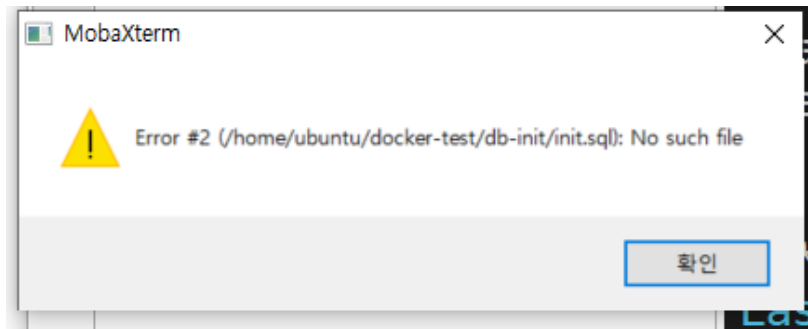
#명령어

```
mkdir docker-test
```

mobaxterm 에서 SFTP를 이용하여 드래그 앤 드롭으로 로컬pc의 파일을
aws의 docker-test폴더에 복사한다.



파일을 드래그앤드롭을 했을 때 아래와 같은 에러 발생 할 수 있다.



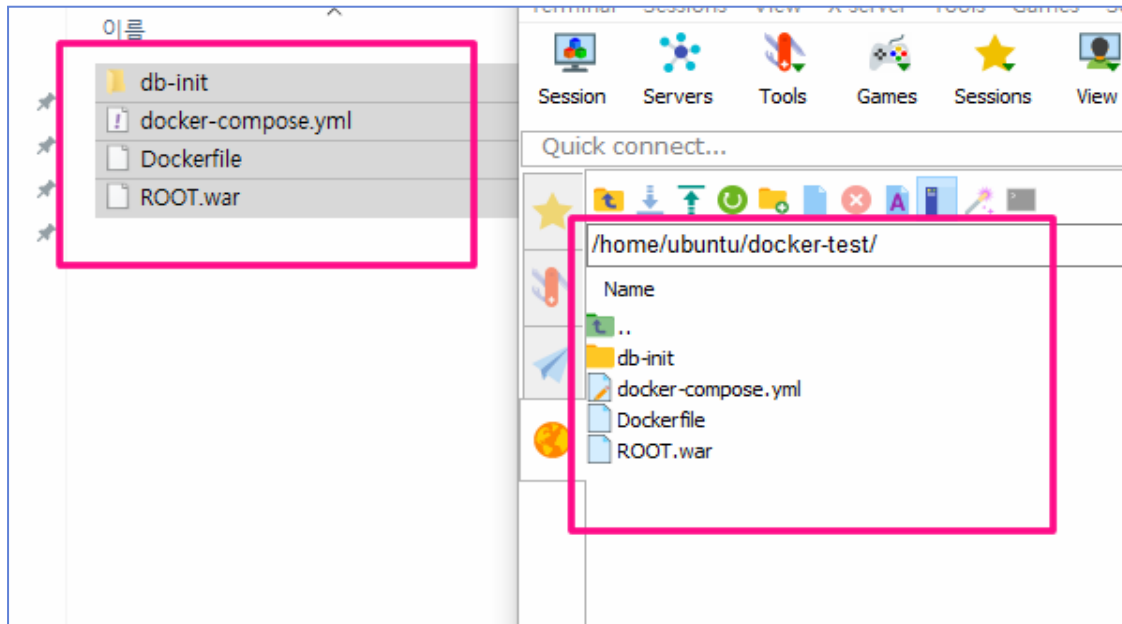
sudo su 명령어 이후 root권한으로 docker-test 디렉토리를 생성했기 때문에 일반 사용자 ubuntu는 이 폴더에 파일을 생성하거나 수정할 권한이 없다.

그래서 소유권을 ubuntu에게 넘겨야, ubuntu 계정으로 로그인한 사용자가 해당 디렉토리에서 자유롭게 작업 가능해진다.

#명령어

```
chown ubuntu:ubuntu /home/ubuntu/docker-test
```

드래그인드랍 다시 해본다.



폴더 docker-test로 이동

#명령어

```
cd docker-test
```

Docker Compose V2부터는 docker-compose 대신 docker compose

#명령어

docker compose version

ls 명령어로 현재 폴더에 docker-compose.yml 있는지 확인해본다.

```
root@ip-172-31-12-26:/home/ubuntu/docker-test# ls
Dockerfile ROOT.war db-init docker-compose.yml
```

#docker-compose실행하기

#명령어

docker compose up --build -d

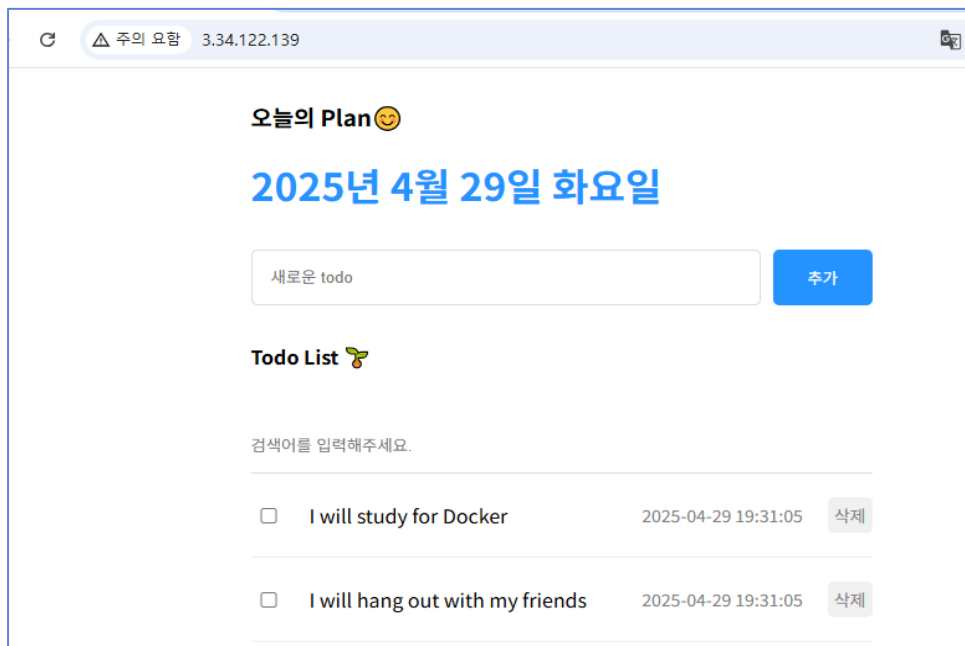
#docker-compose 삭제

#명령어 - 볼륨까지 삭제는 -v 옵션

docker compose down -v

#명령어 - 볼륨까지 삭제안함.

docker compose down



<https://xn--220b31d95hq8o.xn--3e0b707e/>

페이지 21 / 22

△ 주의 요함

heejung.n-e.kr

📄 ☆

오늘의 Plan 😊

2025년 4월 29일 화요일

새로운 todo

추가

Todo List 🌱

검색어를 입력해주세요.

☐

I will study for Docker

2025-04-29 19:31:05

삭제

☐

I will hang out with my friends

2025-04-29 19:31:05

삭제

페이지 22 / 22